SC2002 Object-Oriented Design & Programming
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-23 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1 Objects & Abstraction: Thinking in Objects **1**

1.1 Why Objects? From Messy Reality to Manageable Models 1

1.2 Abstraction, Decomposition, Responsibility 2

1.2.1 Levels of Abstraction 3

1.2.2 Cohesion and Coupling as Judges 3

1.3 Abstraction Criteria and Common Pitfalls 3

1.3.1 Design Exercise: Warm-Up Decomposition 4

1.3.2 Visual Summary: From Requirement to Objects 5

1.4 Chapter Notes 5

1.5 Exercises 5

2 Classes & Encapsulation: From Objects to Blueprints **6**

2.1 Classes as Blueprints 6

2.2 Encapsulation and Access Control 7

2.2.1 Static vs. Instance 8

2.2.2 Immutability and Defensive Copies 8

2.3 Identity, Equality, and String Representation 8

2.4 Constructors, Overloading, and Object Lifecycle 9

2.4.1 Records and Compact Value Types (Java 16+) 9

2.4.2 Common Pitfalls and How Tests Catch Them 9

2.5 Chapter Notes 10

2.6 Exercises 10

3 Inheritance & Polymorphism: Reuse Without Copy-Paste **11**

3.1 Generalisation and Inheritance . . . 11
3.2 Polymorphism and Dynamic Dispatch . . . 12
3.2.1 Abstract Classes and Interfaces . . . 13
3.3 Composition Over Inheritance and LSP . . . 13
3.4 Abstract Classes, Interfaces, and Sealed Hierarchies . . . 14
3.4.1 Overriding Rules and the Fragile Base Class . . . 14
3.4.2 Interfaces with Defaults and Functional Roles . . . 14
3.4.3 Quick Decision Table . . . 15
3.5 Chapter Notes . . . 15
3.6 Exercises . . . 15

4 UML Class & Sequence Diagrams: Visual Contracts 16

4.1 Class Diagrams: Structure at a Glance . . . 16
4.2 Sequence Diagrams: Interaction Over Time . . . 17
4.3 From Diagrams to Code and Back: Consistency . . . 18
4.3.1 Worked Consistency Check: Borrow Use Case . . . 19
4.3.2 Visual Editing Workflow . . . 19
4.4 Chapter Notes . . . 19
4.5 Exercises . . . 20

5 SOLID & Design Principles: From Code That Works to Code That Lasts 21

5.1 Single Responsibility and Open–Closed . . . 21
5.2 Liskov, Interface Segregation, Dependency Inversion . . . 22
5.3 How SOLID Feels in Practice: Smells and Heuristics . . . 23
5.3.1 From Principles to Code Reviews . . . 24
5.3.2 Dependency Inversion in Layers . . . 24
5.4 Chapter Notes . . . 25
5.5 Exercises . . . 25

6 Design Patterns: Reusable Solutions (Strategy, Observer, Factory) 26

6.1 Strategy: Encapsulate What Varies . . . 26

6.2 Observer: Decouple Event Source from Consumers . . . 27
6.3 Factory: Centralise Creation . . . 28
6.4 Choosing and Combining Patterns . . . 29
6.5 Chapter Notes . . . 29
6.6 Exercises . . . 30

7 Exception Handling & Generics: Safety at Compile Time and Runtime 31

7.1 Exceptions: Fail Fast, Fail Clearly . . . 31
7.2 Generics: One Definition, Many Types . . . 33
7.3 Putting It Together: A Type-Safe, Failure-Aware API . . . 33
7.3.1 Erasure Pitfalls Worked . . . 34
7.3.2 Testing Generics and Exceptions . . . 35
7.4 Chapter Notes . . . 35
7.5 Exercises . . . 35

8 Testing & Refactoring: Keeping Code Correct and Clean 36

8.1 Testing: The Safety Net . . . 36
8.2 Refactoring: Change Structure, Preserve Behaviour . . . 38
8.2.1 TDD: Red–Green–Refactor . . . 38
8.3 Refactoring Catalogue in Detail . . . 39
8.3.1 TDD Traces and When Not to Mock . . . 39
8.4 Chapter Notes . . . 40
8.5 Exercises . . . 40

Chapter 1

Objects & Abstraction: Thinking in Objects

> *"Object-oriented design is the art of deciding what to hide."* — Before syntax, we must learn to *see* a system as cooperating objects. This chapter builds that lens from zero: what an object is, why abstraction matters, and how to decompose a problem without writing a line of Java yet.

From zero: no SC1003/SC1007 assumed. We define everything—programs, state, behaviour, abstraction—from first principles. No prior OOP or Java is assumed. Later chapters depend only on what is built here.

Learning Outcomes

After this chapter you will be able to:

(L1) define *object*, *state*, *behaviour*, *identity* and distinguish objects from values;

(L2) explain abstraction, encapsulation boundaries, and decomposition via responsibility assignment;

(L3) contrast procedural (verb-first) vs. object-oriented (noun-first) modelling on the same requirement;

(L4) describe object interaction as *message passing* and identify collaborators and responsibilities (CRC);

(L5) apply abstraction criteria (cohesion, level, information hiding) to judge a proposed model.

1.1 Why Objects? From Messy Reality to Manageable Models

A program solves a real-world problem (library loans, ride-hailing, pixel rendering). Reality is messy: thousands of details, people, rules. An *abstraction* keeps only the details relevant to the problem and hides the rest. An *object* is the unit we use to do this.

Intuition first—picture a vending machine. You see buttons, a coin slot, a tray. You do not see the internal motors. The visible surface is the abstraction; the hidden motors are the implementation. Good objects are

like that: a clear surface, a hidden interior.

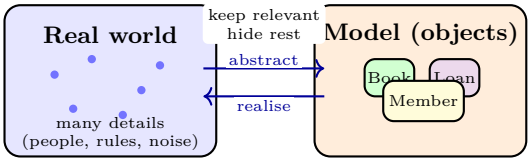


Figure 1.1: Abstraction selects relevant details from reality and packages them as objects (Book, Member, Loan).

Definition 1.1 (Abstraction)**.** An *abstraction* is a simplified representation that retains properties relevant to a purpose and suppresses irrelevant detail. Good abstractions have high *cohesion* (one purpose) and low *coupling* (few dependencies). Abstraction is not vague—it is selective.

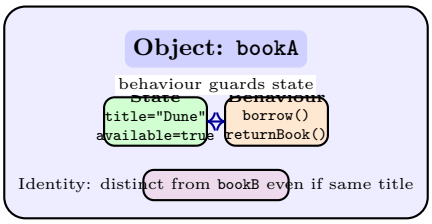


Figure 1.2: Every object has *identity* (which object), *state* (fields), and *behaviour* (methods). Behaviour is the only way to observe or change state.

Definition 1.2 (Object, State, Behaviour, Identity)**.** An *object* is an encapsulated entity with (i) *identity* (a distinct existence; two objects with equal fields are not the same object), (ii) *state* (the values of its attributes at a moment), and (iii) *behaviour* (operations it can perform, typically by reading/writing its state and collaborating with others).

Values vs. objects: `42` and `"hi"` are *values*—no identity beyond equality. A `Book` or `BankAccount` is an object—identity matters (your account $\neq$ my account even with same balance). Confusing the two is a common early error.

1.2 Abstraction, Decomposition, Responsibility

To design is to decompose. Given "build a library system," we ask: what *things* exist, what does each *know* and *do*, and who does it *talk to*?

A classic lightweight tool is **CRC cards** (Class–Responsibility–Collaborator): for each candidate object write its responsibilities (what it must achieve) and collaborators (who it needs). Example: `Loan` — responsibility "track due date, detect overdue" — collaborators `Book`, `Member`.

Example 1.1 (Library decomposition—first cut)**.** Nouns in the brief: *Book, Member, Loan, Librarian.* Verbs: *borrow, return, reserve, fine.* Candidate objects: `Book` knows its availability; `Loan` knows its due date; `Member` knows its loans. Librarian is an *actor* (human), not necessarily an object—model what the system must remember, not every person in the world.

Information hiding: each object publishes an *interface* (what you may ask it) and hides its representation (how it stores it). That lets us change one object's internals without breaking callers—the payoff of abstraction.

1.2.1 Levels of Abstraction

Abstractions nest. At the top, "Library lends Books." One level down, "Loan has a due date, Member has a borrowing limit." Further down, "Due date is a `LocalDate`." Each level hides detail of the one below. Jumping levels (e.g., calculating fine inside UI code) breaks encapsulation and scatters knowledge.

1.2.2 Cohesion and Coupling as Judges

Cohesion asks "does this object do one thing well?" A `LoanCalculator+Printer+Saver` class has low cohesion. *Coupling* asks "how many others does this object depend on?" High coupling means one change ripples everywhere. Prefer many cohesive, loosely coupled objects over one god object.

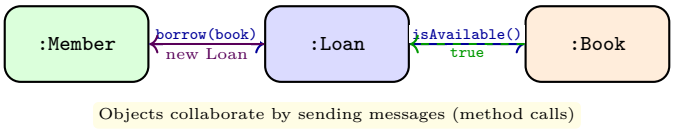


Figure 1.3: Message passing: a `Member` asks to borrow; the `Loan` checks the `Book`'s availability before creation.

Remark 1.1 (Procedural vs. OO—verb-first vs. noun-first). Procedural: "steps to borrow a book" — one procedure reads/writes global arrays. OO: "which object should be responsible for what?" — `Book` guards availability, `Loan` guards dates, `Member` guards limits. When requirements change ("add reservations"), OO changes are local; procedural changes ripple. The shift is not syntax but where knowledge lives.

Listing 1.1: Procedural vs. OO sketch—same requirement, different homes for logic.

```
// Procedural (data + separate functions) -- logic scattered
boolean borrowBook(int bookId, int memberId) {
    if (!books[bookId].available) return false;
    if (memberLoans[memberId].size() >= MAX) return false;
    // ... update arrays directly -- invariants checked nowhere centrally
}
// OO (behaviour lives with data) -- each object guards its invariant
class Book { boolean isAvailable() { ... } void markBorrowed() { ... } }
class Member { boolean canBorrow() { return loans.size() < MAX; } }
class Loan { static Loan create(Member m, Book b) {
    if (!b.isAvailable() || !m.canBorrow()) throw new IllegalStateException();
    b.markBorrowed(); Loan ln = new Loan(m,b); m.addLoan(ln); return ln;
}}
```

1.3 Abstraction Criteria and Common Pitfalls

How do you judge an abstraction? Three tests:

1. Does it hide a design decision? If the caller would need to change when you change representation, the abstraction leaks. Example: exposing `books: Book[]` forces callers to know it is an array; switching to a `Map<Isbn,Book>` breaks them. Hide behind `findByIsbn(Isbn)` instead.

2. Is it at a consistent level? A `Library` method that both checks borrowing rules and formats a receipt mixes policy with presentation. Split levels: policy (`LoanPolicy`), persistence (`Repository`), presentation (`ReceiptFormatter`).

3. Can you state its invariant in one sentence? "A `Book` is available iff no active `Loan` references it" is crisp. "`Library` does everything" states no invariant—a warning sign.

Listing 1.2: Leaky vs. encapsulated—caller dependency on representation.

Example 1.2 (Leaky abstraction—and the fix)**.**

```
// Leaky: caller knows loans are a List and manipulates it directly
library.getLoans().add(new Loan(member, book)); // invariant not checked!
// Encapsulated: Library enforces the invariant
library.borrow(memberId, bookId); // checks availability, limit, creates Loan atomically
```

Remark 1.2 (Noun-verb heuristic—use, then validate). Underline nouns/verbs in the requirements to seed candidates, then validate each candidate with CRC: does it have a crisp responsibility and a small collaborator set? Discard candidates that are really attributes (`dueDate`) or roles (`borrower` is a `Member` playing a role).

1.3.1 Design Exercise: Warm-Up Decomposition

Walk through a tiny system end-to-end to cement the method.

Requirement. "A vending machine sells snacks. Customers insert coins, select a snack, receive change." Nouns: `Customer`, `Coin`, `Snack`, `VendingMachine`, `Transaction`. Verbs: `insert`, `select`, `dispense`, `refund`. CRC first draft:

- `Snack`: knows price, stock; collaborator none.
- `VendingMachine`: knows inventory, coin store; collaborators `Snack, Transaction`.
- `Transaction`: knows amount paid, change due; collaborators `Coin`.

Notice `Customer` is an actor outside the system—no class needed unless we track customer accounts. The boundary (what is inside vs. outside) is the first design decision.

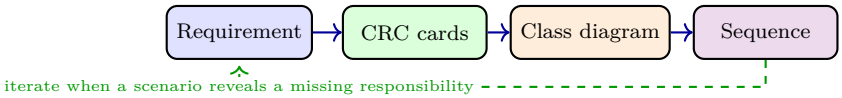
Listing 1.3: CRC to code—one object per responsibility, no god class.

```
class Snack { private String name; private int priceCents; private int stock;
    boolean inStock(){ return stock>0; } void dispense(){ stock--; } }
class Transaction { private int paid, price;
    int changeDue(){ return paid - price; } boolean paidEnough(){ return paid>=price; } }
```

This two-object split already enforces an invariant: stock never goes negative because `dispense` is only called after an `inStock` check inside `VendingMachine`.

1.3.2 Visual Summary: From Requirement to Objects

The object-oriented workflow is iterative: (1) harvest nouns/verbs, (2) draft CRC cards, (3) sketch a class diagram, (4) walk through scenarios with sequence diagrams, (5) refine. Each step hides one level of detail so the next step can proceed without drowning.



Iteration is the point—first drafts are wrong, and CRC/sequence walks are the cheapest way to discover it, before any Java is written.

1.4 Chapter Notes

Abstraction and information hiding are Parnas (1972); CRC cards are Beck & Cunningham (1989). "Message passing" terminology is Smalltalk/Kay; Java/C++ realise it as method calls. Cohesion/coupling metrics originate with Constantine & Yourdon. The noun-verb heuristic for finding objects is useful but fallible—always validate with responsibilities.

1.5 Exercises

E1.1 **Object vs. value.** Give two domain examples where identity matters (must be objects) and two where only equality matters (values). What goes wrong if you model a value as an object and vice versa?

E1.2 **CRC drill.** For a ride-hailing app with `Rider, Driver, Trip, Payment`, write CRC cards (responsibilities + collaborators) for each. Which responsibilities, if misplaced, would create high coupling?

E1.3 **Abstraction quality.** A teammate models `Library` with one method `doEverything(String command)`. Diagnose using cohesion, coupling, and information hiding. Propose a refactored decomposition and justify each object's boundary.

E1.4 **Message trace.** Draw the message sequence for "member returns a book late and incurs a fine." Show which object computes the fine and why—argue from responsibility assignment, not implementation convenience.

E1.5 **Procedural to OO.** Rewrite a procedural `transfer(fromId, toId, amount)` that manipulates a global `balances[]` array into an OO design with `Account` objects. Where does the invariant `balance` $\geq$ `0` now live, and how is it preserved?

Chapter 2

Classes & Encapsulation: From Objects to Blueprints

> *"A class is a cookie cutter; objects are the cookies."* — One object is an example; a *class* is the recipe. Encapsulation decides what the recipe reveals and what it hides. This chapter turns the object intuition from Ch. 1 into code-level discipline.

From zero: built on Ch. 1 only. We define *class*, *field*, *method*, *constructor*, and *visibility* from scratch. Java is our notation but the ideas port to C#, Python, C++. No prior Java assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish *class* (type/blueprint) from *object* (instance) and explain instantiation;

(L2) declare fields, constructors, accessors/mutators, and enforce invariants via encapsulation;

(L3) apply access modifiers (`private`, package, `protected`, `public`) and `static` vs. instance;

(L4) reason about object identity vs. equality (`==` vs. `equals`) and `toString` contracts;

(L5) justify information hiding using coupling and change-impact arguments.

2.1 Classes as Blueprints

In Ch. 1 an *object* was identity + state + behaviour. A *class* packages the shared structure for many such objects: it declares what fields every instance has, what methods it offers, and how instances are created.

Definition 2.1 (Class vs. Object)**.** A *class* is a type declaration defining fields (state shape), methods (behaviour), and constructors (creation protocol). An *object* (instance) is a runtime entity of that class with its own field values and identity. `new Book(...)` allocates an instance.

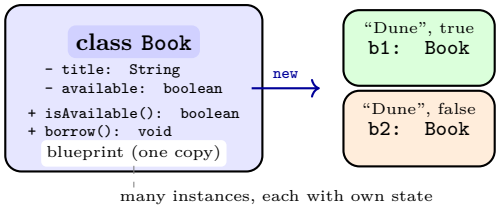


Figure 2.1: Class as blueprint: one `Book` class, many `Book` instances each with independent state.

Fields, methods, constructors. Fields hold state; methods operate on it; constructors initialise a fresh instance and establish its *invariant* (a property that should always hold, e.g., `title != null`).

Listing 2.1: Minimal class with invariant and constructor.

```
public class Book {
    private String title;       // hidden state
    private boolean available;  // invariant: title != null
    public Book(String title) {
        if (title == null) throw new IllegalArgumentException("title required");
        this.title = title;
        this.available = true;  // new books are available
    }
    public boolean isAvailable() { return available; }
    public void markBorrowed() {
        if (!available) throw new IllegalStateException("already borrowed");
        available = false;
    }
}
```

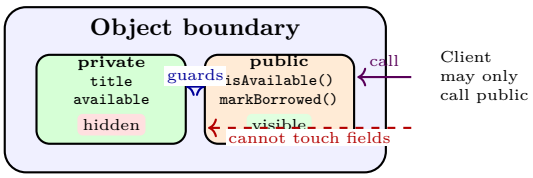


Figure 2.2: Encapsulation: private state is hidden behind a public interface. All state changes go through methods that can enforce invariants.

2.2 Encapsulation and Access Control

Encapsulation bundles data and the methods that maintain it, and hides the data. Java enforces this with visibility:

Modifier	class	package	subclass	world
`private`	✓			
package (default)	✓	✓		
`protected`	✓	✓	✓	
`public`	✓	✓	✓	✓

Rule of thumb: make fields `private` by default; expose behaviour, not data. Provide getters only when callers

truly need the information, and prefer telling objects what to do over asking for data and deciding externally ("Tell, Don't Ask").

2.2.1 Static vs. Instance

`static` members belong to the *class* (one copy): constants, factories, counters of instances. Instance members belong to each *object*. Confusing them is a common bug: `static int nextId` is shared; `int id` is per-object.

2.2.2 Immutability and Defensive Copies

An *immutable* class has no mutators and all fields `final`—instances cannot change after construction (e.g., `String`, `LocalDate`). Immutability eliminates aliasing bugs. When a class holds a mutable field (e.g., `Date` or `List`), return defensive copies or wrap with `Collections.unmodifiableList`.

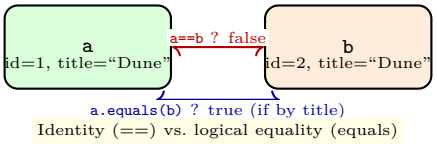


Figure 2.3: Identity (`==`) checks same object; `equals` checks logical equivalence as defined by the class.

2.3 Identity, Equality, and String Representation

Java has two equalities: `==` (same heap object) and `equals` (logical equality). Default `equals` is identity—override it when value-semantics matter, and always override `hashCode` consistently: equal objects must have equal hashes.

Listing 2.2: Equality contract and toString—value semantics done right.

```
public final class Isbn {
    private final String code;
    public Isbn(String code) { this.code = code; }
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Isbn)) return false;
        return code.equals(((Isbn)o).code);
    }
    @Override public int hashCode() { return code.hashCode(); }
    @Override public String toString() { return "Isbn[" + code + "]"; }
}
```

Remark 2.1 (Common pitfall). Storing mutable objects in `HashSet`/`HashMap` and then mutating fields that affect `hashCode` corrupts the collection. Prefer immutable keys.

2.4 Constructors, Overloading, and Object Lifecycle

A constructor's job is to establish the invariant from birth. If construction can fail, fail fast with an exception—never create a "zombie" object that is half-initialised.

Constructor chaining. One constructor should do the work; others delegate via `this(...)` so validation lives in one place.

Listing 2.3: Constructor chaining—single validation point.

```
public class Member {
    private final String id;
    private final String name;
    private final int maxLoans;
    public Member(String id, String name){ this(id, name, 5); }
    public Member(String id, String name, int maxLoans){
        if (id==null||name==null) throw new IllegalArgumentException("id/name required");
        if (maxLoans<1||maxLoans>10) throw new IllegalArgumentException("maxLoans 1..10");
        this.id=id; this.name=name; this.maxLoans=maxLoans;
    }
}
```

Overloading vs. overriding (preview of Ch. 3). Overloading is same name, different parameters, resolved at compile time. It is convenient for constructors and utility methods but can surprise with autoboxing/null. Overriding (runtime, via inheritance) is covered fully in Ch. 3.

Lifecycle: creation → use → GC. After `new`, the object is reachable while referenced; when unreachable it becomes eligible for garbage collection. No explicit `free` is needed. Reachability explains memory leaks: a forgotten observer list entry keeps an object alive forever—see Ch. 6.

2.4.1 Records and Compact Value Types (Java 16+)

For immutable data carriers, `record` generates constructor, accessors, `equals/hashCode/toString` automatically. Ideal for value objects like `Isbn`, `Money`, `IsbnRange`. Use a class when you need mutability or invariant enforcement beyond the canonical constructor.

2.4.2 Common Pitfalls and How Tests Catch Them

- *Exposing mutable internals.* Return an unmodifiable view (e.g., `Collections.unmodifiableList(books)`) or a copy; otherwise `library.getBooks().clear()` destroys state. A test that mutates the returned list and checks the library still has its books catches this.
- *Inconsistent equals/hashCode.* Two equal `Isbn` objects landing in different hash buckets breaks `HashSet` lookup. Test: add equal objects, assert `set.size()==1` and `set.contains(copy)`.

- *Forgetting defensive copies of Date/List.* Caller mutates the object after passing it in. Fix: copy on entry (`this.dueDate = LocalDate.from(dueDate)`) and on exit; `LocalDate` is already immutable, which is why modern Java prefers it over `Date`.

Listing 2.4: Defensive copy for a mutable field (contrast with immutable LocalDate).

```
class Loan {
    private final LocalDate dueDate; // immutable -- no copy needed
    private final List<String> notes; // mutable -- must copy
    Loan(LocalDate dueDate, List<String> notes){
        this.dueDate = dueDate; // LocalDate is immutable
        this.notes = List.copyOf(notes); // defensive copy + unmodifiable
    }
    List<String> getNotes(){ return notes; } // already unmodifiable
}
```

2.5 Chapter Notes

Encapsulation is Parnas (1972); "Tell, Don't Ask" is Cunningham. Effective Java (Bloch) Ch. 4 covers access control, immutability, and the equals/hashCode contract. Java's package-private default is a deliberate middle ground between `private` and `public`.

2.6 Exercises

E2.1 **Encapsulation audit.** A `BankAccount` exposes `public double balance`. Give a concrete failure this allows, then redesign with `private` + methods `deposit`, `withdraw` that preserve `balance` $\geq$ `0` and explain how coupling drops.

E2.2 **Constructor invariants.** Write a `Member` class with `maxLoans` invariant $1 \leq$ `maxLoans` ≤ 10. Show constructor validation, a getter, and why no setter is needed. What test would catch a missing check?

E2.3 **Static vs. instance.** A student puts `private String title` as `static` in `Book`. Describe the observable failure with two `Book` instances and fix it. When *should* a field be `static`?

E2.4 **Identity vs. equality.** Two `Isbn("978-0-06-025492-6")` objects: compare `==` and `equals`. Explain why `hashCode` must be overridden and give a failing `HashSet` scenario if it is not.

E2.5 **Defensive copies.** `Library` holds `private List<Book> books` and returns it directly. Show how a caller can break the invariant, then fix with either a copy or unmodifiable view and discuss trade-offs.

Chapter 3

Inheritance & Polymorphism: Reuse Without Copy-Paste

> *"Inheritance is not for reuse—reuse is a consequence. Inheritance is for polymorphism."* — Copy-paste is the enemy; inheritance and polymorphism let us vary behaviour while honouring a common contract.

From zero: built on Ch. 1–2. We define *inheritance*, *subtyping*, *overriding*, *dynamic dispatch*, and *polymorphism* from scratch. No prior inheritance experience assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) model generalisation/specialisation with inheritance and distinguish `extends` vs. subtyping;

(L2) override methods correctly, apply `super`, and explain dynamic dispatch (late binding);

(L3) contrast overloading (compile-time) vs. overriding (runtime) and abstract classes vs. interfaces;

(L4) apply Liskov Substitution Principle informally (subtype must be usable as supertype);

(L5) decide when to prefer composition over inheritance.

3.1 Generalisation and Inheritance

Many domain concepts share structure: `Book`, `EBook`, `AudioBook` all have title/author/id; e-books add download links, audiobooks add duration. Rather than duplicating fields, we factor the common part into a *superclass*.

Definition 3.1 (Inheritance and Subtyping)**.** *Inheritance* is a language mechanism where a subclass reuses/extends a superclass's members. *Subtyping* is the type relationship: every instance of the subclass is also an instance of the supertype ($S <: T$), so it may be used wherever T is expected.

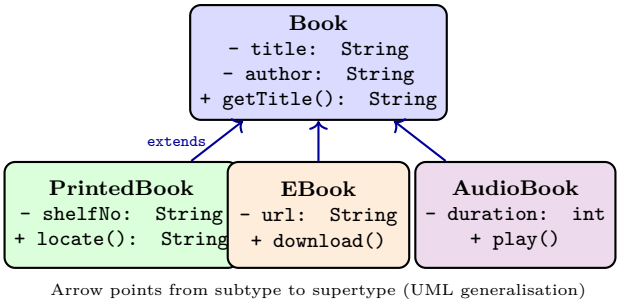


Figure 3.1: Inheritance factors common state/behaviour into a superclass; subclasses specialise by adding fields/methods or overriding behaviour.

Java: `class EBook extends Book { ... }`. The subclass inherits accessible fields/methods, may add new ones, and may *override* inherited methods with a more specific implementation. Constructors are not inherited—each class constructs its own part via `super(...)`.

Listing 3.1: Inheritance with constructor chaining and overriding.

```
public class Book {
    protected String title; protected String author;
    public Book(String title, String author) { this.title=title; this.author=author; }
    public String description() { return title + " by " + author; }
}
public class EBook extends Book {
    private String url;
    public EBook(String t, String a, String url) { super(t,a); this.url=url; }
    @Override public String description() { return super.description() + " [eBook: " + url
        + "]"; }
    public void download() { /* ... */ }
}
```

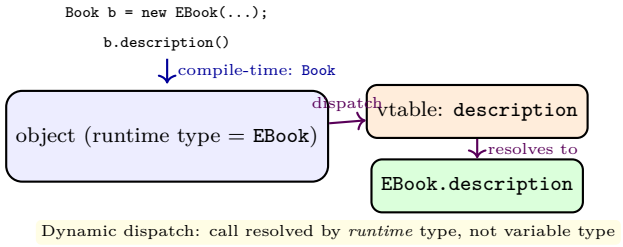


Figure 3.2: Dynamic dispatch (late binding): the method invoked depends on the object's actual class at runtime.

3.2 Polymorphism and Dynamic Dispatch

Polymorphism ("many forms") means one interface, many implementations. In Java, a variable of type `Book` may hold any subtype instance; calling `description()` executes the subtype's override. This is *dynamic dispatch* (late binding).

Why it matters: code that works with `Book` need not know which subtype it has. Adding `AudioBook` requires no changes to loan logic that calls `book.description()`—open for extension, closed for modification (preview of Ch. 5).

Overloading vs. overriding. Overloading: same name, different parameters, resolved at compile time. Overriding: same signature, subclass replaces behaviour, resolved at runtime. Missing `@Override` hides bugs silently—always annotate.

3.2.1 Abstract Classes and Interfaces

An *abstract* class cannot be instantiated; it may declare abstract methods (no body) that subclasses must implement. An *interface* is a pure contract (no state, only method signatures + default/static helpers since Java 8). Prefer interfaces for "can-do" roles (e.g., `Borrowable`, `Payable`); use abstract classes when subclasses share state and common implementation.

Listing 3.2: Interface as contract; multiple behaviours via polymorphism.

```
public interface Borrowable { boolean isAvailable(); void markBorrowed(); }
public abstract class LibraryItem implements Borrowable {
    protected String id; protected boolean available = true;
    public boolean isAvailable() { return available; }
    public abstract String description(); // each subtype describes differently
}
public class Book extends LibraryItem {
    private String title;
    public Book(String id, String t) { this.id=id; this.title=t; }
    public void markBorrowed() { available=false; }
    public String description() { return "Book: " + title; }
}
```

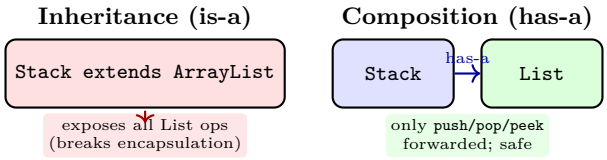


Figure 3.3: Prefer composition over inheritance when you want reuse without exposing the reused type's full interface.

3.3 Composition Over Inheritance and LSP

Inheritance couples subclass to superclass implementation. If the superclass changes, subclasses break (fragile base class problem). Guideline: use inheritance only for true *is-a* with behavioural substitutability.

Definition 3.2 (Liskov Substitution Principle — informal)**.** If S is a subtype of T, objects of T should be replaceable by objects of S without breaking correctness. Overridden methods must honour the supertype's contract (preconditions no stronger, postconditions no weaker).

Classic violation: `Square extends Rectangle` with `setWidth` that also sets height—code expecting independent width/height breaks. Fix: do not force that hierarchy; model `Shape` with separate subtypes or use composition.

Remark 3.1 (Checklist: inheritance or composition?). Is it truly is-a and will callers use the subtype as the supertype? Does the subclass need to override behaviour (polymorphism)? If either answer is no, prefer composition (has-a) with delegation.

3.4 Abstract Classes, Interfaces, and Sealed Hierarchies

When to use which. Start with an interface for a role; introduce an abstract class when multiple implementors share state or common code. Java lets a class implement many interfaces but extend one class—so prefer interfaces for flexibility, abstract classes for code sharing.

Template Method (preview of Ch. 6). An abstract class can define a skeleton algorithm calling abstract steps that subclasses fill in—the classic way to share control flow while varying details.

Listing 3.3: Template Method via abstract class.

```
public abstract class LoanPolicy {
    public final boolean canBorrow(Member m, Book b){ // skeleton
        return isAvailable(b) && withinLimit(m) && noOverdue(m);
    }
    protected abstract boolean isAvailable(Book b);
    protected abstract boolean withinLimit(Member m);
    protected abstract boolean noOverdue(Member m);
}
```

Sealed classes (Java 17+). `sealed class Shape permits Circle, Rectangle` lets the compiler know all subtypes, enabling exhaustive `switch` with no default—a safer closed hierarchy when the set of variants is fixed.

3.4.1 Overriding Rules and the Fragile Base Class

Overriding must match signature and not narrow visibility; `@Override` makes the compiler enforce it. The fragile-base-class problem: a superclass change (e.g., adding a method that happens to match a subclass method) can silently alter behaviour. Mitigations: favour composition (Fig. 3.3), make classes `final` by default and document extension points, prefer interfaces.

3.4.2 Interfaces with Defaults and Functional Roles

Since Java 8, interfaces may carry `default` methods (shared behaviour) and `static` helpers. Use defaults sparingly to avoid the diamond problem; prefer abstract classes when defaults need state.

Listing 3.4: Interface defaults—shared behaviour without a base class.

```
interface Borrowable {
    boolean isAvailable();
    void markBorrowed();
    default void borrowOrThrow(){
        if (!isAvailable()) throw new IllegalStateException("not available");
```

```
6           markBorrowed();
7       }
8   }
```

Functional interfaces (`Predicate<T>, Function<T,R>`) plus lambdas often replace single-method Strategy hierarchies with less ceremony—the *idea* (pluggable behaviour) stays, the boilerplate shrinks. Choose explicit Strategy classes when the behaviour is complex or has its own state/tests.

3.4.3 Quick Decision Table

Need	Prefer	Why
Share state + common code	abstract class	fields + constructor
Pure capability, many implementors	interface	multiple implementation
Closed small variant set	sealed class/interface	exhaustive switch
One-method pluggable behaviour	functional interface / lambda	less boilerplate

3.5 Chapter Notes

Liskov's principle is Liskov (1987) / Liskov & Wing (1994). Bloch (Effective Java) Item 18: "Favour composition over inheritance." Java's single-class inheritance + multiple-interface implementation balances reuse and complexity; other languages differ (C++ multiple inheritance, Python MRO).

3.6 Exercises

E3.1 **Override vs. overload.** Given `void foo(Book)` and `void foo(EBook)` overloads vs. `description()` override, predict which is chosen at compile vs. runtime and write a test that demonstrates the difference.

E3.2 **Dispatch trace.** With `Book b = new EBook(...); b.description();` explain step-by-step what the compiler checks and what the runtime does. What if `description` were `static`?

E3.3 **Abstract vs. interface.** Model `Payable` (Shenanigan: trip fare, library fine, e-commerce order). Should it be an abstract class or interface? Justify with state-sharing and multiple-inheritance needs.

E3.4 **LSP violation.** Show why `Square extends Rectangle` violates LSP with a code snippet that passes for `Rectangle` but fails for `Square`. Propose a corrected hierarchy.

E3.5 **Composition refactor.** Refactor `Stack extends ArrayList` to composition. List which methods you expose, how you delegate, and why this is safer under future `ArrayList` changes.

Chapter 4

UML Class & Sequence Diagrams: Visual Contracts

"A diagram is worth a thousand lines—if it has a legend." — UML is not decoration; it is a precise visual contract between designer and implementer. This chapter teaches you to read and draw the two diagrams you will use most: class and sequence.

From zero: built on Ch. 1–3. We define UML notation from first principles—no prior modelling assumed. Class diagrams capture *structure*; sequence diagrams capture *behaviour over time*. Both map directly to Java.

Learning Outcomes

After this chapter you will be able to:

(L1) read and draw UML class diagrams: classes, attributes, operations, visibility, associations, multiplicity, generalisation, dependency, composition/aggregation;

(L2) read and draw UML sequence diagrams: lifelines, messages (sync/async/return), alt/opt/loop fragments, creation/destruction;

(L3) translate a class diagram to Java skeletons and a sequence diagram to method-call order;

(L4) choose the right diagram for a question (structure vs. interaction) and keep them consistent;

(L5) critique a UML diagram for completeness, multiplicity errors, and directionality.

4.1 Class Diagrams: Structure at a Glance

A class diagram shows the *static* structure: what types exist, what each knows, and how they relate. Notation is compact but precise.

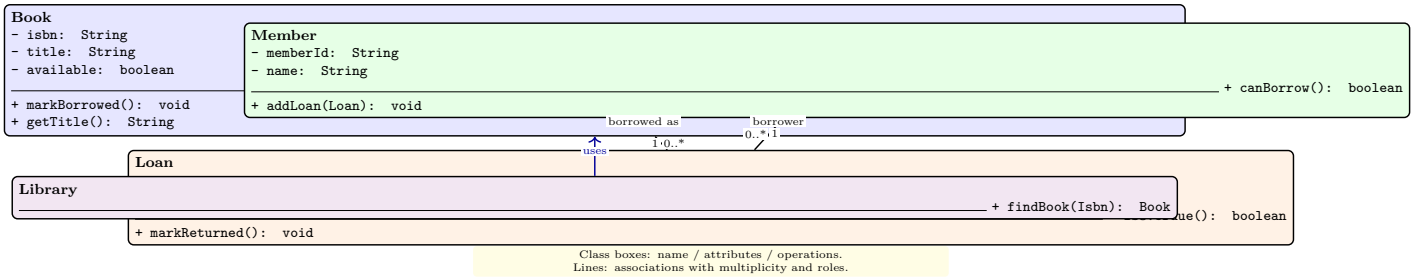


Figure 4.1: Library class diagram: `Book`–`Loan`–`Member` associations with multiplicities; `Library` depends on `Book`.

Class box. Three compartments: name (bold, abstract in *italics*), attributes (`visibility name: Type`), operations (`visibility name(params): ReturnType`). Visibility: `+` public, `-` private, `#` protected, `~` package.

Relationships.

- *Association* (solid line): "knows about." Label with role names and *multiplicity*: `1`, `0..1`, `*`, `1..*`, `0..*`.
- *Composition* (filled diamond): part dies with whole (`Library 1` ♦– `Loan *`—if Library deleted, Loans deleted).
- *Aggregation* (hollow diamond): part may outlive whole.
- *Generalisation* (hollow triangle arrow): inheritance.
- *Dependency* (dashed arrow): "uses temporarily" (parameter, local variable, static call).

Listing 4.1: From class diagram to Java—multiplicity guides the collection type.

```
//  Member 1 -- 0..* Loan  => Member holds a collection of Loans
class Member {
    private String memberId;
    private List<Loan> loans = new ArrayList<>();
    public boolean canBorrow() { return loans.size() < 5; }
    public void addLoan(Loan l) { loans.add(l); }
}
class Loan {
    private Member borrower;  //  Loan * -- 1 Member
    private Book book;        //  Loan * -- 1 Book
    private LocalDate dueDate;
    public boolean isOverdue() { return LocalDate.now().isAfter(dueDate); }
}
```

4.2 Sequence Diagrams: Interaction Over Time

A sequence diagram shows *how* objects collaborate for one scenario. Time flows downwards; each vertical dashed line is a *lifeline*; horizontal arrows are *messages*.

Message kinds: synchronous call (filled arrowhead, caller blocks), asynchronous (open arrowhead, fire-and-forget), return (dashed), creation (`create` with new lifeline), destruction (`X` at end). Fragments: `alt` (if/else),

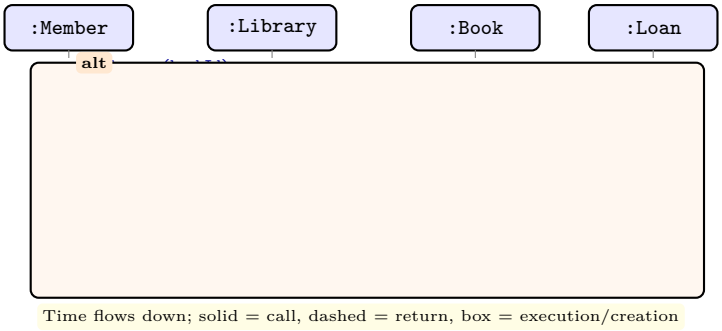


Figure 4.2: Sequence diagram for `borrow`: `Member` → `Library` → `Book` checks, then `Loan` creation. Add `alt` for failure branches.

`opt` (if), `loop` (while/for), `ref` (reference another diagram).

Remark 4.1 (Class vs. sequence—when to use which). Use a class diagram to answer "what types and relations exist?" (structure). Use a sequence diagram to answer "in what order do they interact for this use case?" (behaviour). Keep them consistent: every message's target operation must appear in the class diagram.

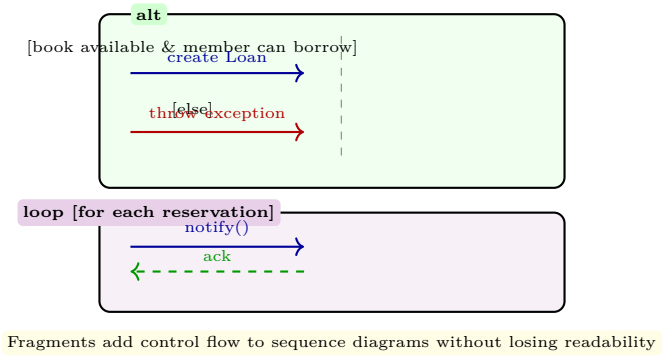


Figure 4.3: Fragments: `alt` for branches, `loop` for iteration (add guards in brackets).

4.3 From Diagrams to Code and Back: Consistency

UML earns its keep only if diagrams and code agree. Two consistency rules:

Class ↔ sequence agreement. Every lifeline in a sequence diagram must correspond to a class (or actor) in the class diagram; every message must correspond to an operation declared on the target's class. A mismatch is a defect, not a "detail for later."

Multiplicity guides implementation. `1` → single reference, `0..1` → `Optional<T>` or nullable reference, `*` → `List<T>` or `Set<T>`. Navigability arrows tell you where the reference lives: if only `Member` → `Loan` is navigable, only `Member` holds a collection.

Listing 4.2: Navigability decides field placement.

```
// Association  Member 1 ---- 0..* Loan  (navigable Member -> Loan only)
class Member { private List<Loan> loans = new ArrayList<>(); } // Member holds it
class Loan { /* no back-pointer to Member unless navigable the other way */ }

```

```
// If navigable both ways, both hold references (keep in sync via one owner method)
class Loan2 { private Member borrower; private Book book; }
```

Tooling and levelling. Keep diagrams at the level of the audience: analysis diagrams omit private helpers; design diagrams include them. Generate class skeletons from UML (PlantUML, IntelliJ) and round-trip carefully—never let the diagram drift from the code after the first sprint.

Remark 4.2 (Diagram smells). God-class box with 30 operations, associations without multiplicity, sequence diagrams that never show `alt` for error paths, and lifelines labelled with class names instead of roles (`:Book` vs. `Book`) are all review flags.

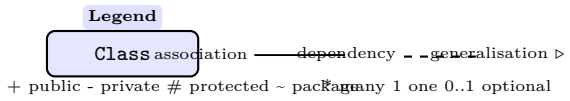
4.3.1 Worked Consistency Check: Borrow Use Case

Given the class diagram in Fig. 4.1 and the sequence in Fig. 4.2, verify: (i) `Library.borrow` exists on `Library`, (ii) `Book.isAvailable` exists on `Book`, (iii) `Member.canBorrow` exists on `Member`, (iv) `Loan` creation parameters match its constructor, (v) multiplicities allow the new `Loan` (member has < 5 loans). Any failure is a modelling defect caught before coding.

Review checklist for a class diagram. Are all associations labelled with multiplicity and role? Is navigability correct (who holds the reference)? Are generalisations substitutable (LSP)? Are dependencies dashed and associations solid? For a sequence diagram: do `alt` branches cover error paths? Are lifelines created/destroyed at the right time? Do guards read as boolean expressions?

4.3.2 Visual Editing Workflow

Draw class diagrams top-down (most stable abstractions at the top, details at the bottom) and sequence diagrams left-to-right in call order. Colour by layer (blue=domain, green=service, orange=data) so a reader can see architectural boundaries at a glance. Keep a one-page legend (visibility symbols, line styles, fragment names) on the first UML page of any document.



A legend removes ambiguity and costs one small TikZ—always include it in hand-ins.

4.4 Chapter Notes

UML 2.5.1 is the current standard (OMG). We cover only class + sequence—the highest-ROI pair. State-machine and activity diagrams are useful complements (SC2006/advanced modelling). Tools: PlantUML, IntelliJ UML, Visual Paradigm; any tool that exports clear, colour-legended diagrams suffices.

4.5 Exercises

E4.1 **Class diagram reading.** Given the library class diagram in Fig. 4.1, list every association's multiplicity, role names, and navigability. Write the Java field declarations that realise each association.

E4.2 **Composition vs. aggregation.** Model `Order`–`OrderLine` and `Department`–`Professor`. Which should be composition and which aggregation? Justify via lifecycle (does part die with whole?).

E4.3 **Sequence to code.** Translate Fig. 4.2 into a Java `Library.borrow(memberId, bookId)` skeleton that returns a `Loan` or throws. Show where each message becomes a method call.

E4.4 **Fragment modelling.** Draw a sequence diagram for "member reserves an unavailable book; when the book is returned the next reserver is notified." Use `alt` and `loop` fragments.

E4.5 **Consistency check.** A class diagram shows `Loan` with no `isOverdue()` but the sequence diagram calls it. A teammate says "it's fine, we will add it later." Explain why this inconsistency is a defect and how tool or review can catch it.

Chapter 5

SOLID & Design Principles: From Code That Works to Code That Lasts

> *"You cannot design well without principles; you cannot follow principles without judgement."* — SOLID is five interlocking principles that turn "it works" into "it stays workable." Each principle names one way code degrades—and the habit that prevents it.

From zero: built on Ch. 1–4. Each SOLID principle is motivated from a concrete code smell you have already seen (god class, copy-paste, fragile override). No prior design-principles assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) state each SOLID principle and the symptom it cures (SRP, OCP, LSP, ISP, DIP);

(L2) apply cohesion/coupling, DRY, and encapsulation to judge a design;

(L3) refactor a god class, a switch-on-type, and a fat interface using SOLID;

(L4) explain dependency inversion and how interfaces decouple high-level policy from low-level detail;

(L5) trade off principles against simplicity (YAGNI, KISS) and avoid over-engineering.

5.1 Single Responsibility and Open–Closed

SRP—A class should have one reason to change. A `Report` that formats, prints, and saves has three reasons to change (layout, printer driver, file format). Split into `Report`, `ReportFormatter`, `ReportPrinter`, `ReportRepository`—each with high cohesion.

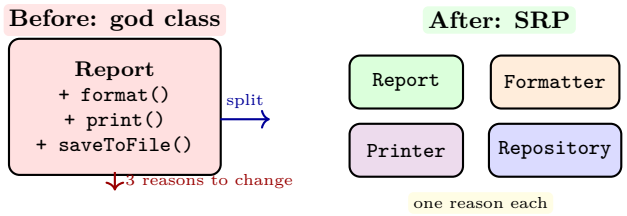


Figure 5.1: SRP: split a god class so each class has one axis of change.

OCP—Open for extension, closed for modification. New behaviour should be added by adding new code, not editing old code. The mechanism is polymorphism: depend on an abstraction, plug in new implementations.

Listing 5.1: OCP via polymorphism—no switch, no edit to add a new shape.

```
// Violates OCP: every new shape edits this method
double area(Object s) {
    if (s instanceof Circle c) return Math.PI*c.r()*c.r();
    if (s instanceof Rectangle r) return r.w()*r.h();
    throw new IllegalArgumentException();
}
// OCP: add a new Shape subtype; AreaCalculator never changes
interface Shape { double area(); }
class Circle implements Shape { double r; public double area(){return Math.PI*r*r;}}
class AreaCalculator { double total(List<Shape> ss){ return ss.stream().mapToDouble(Shape
    ::area).sum();}}
```

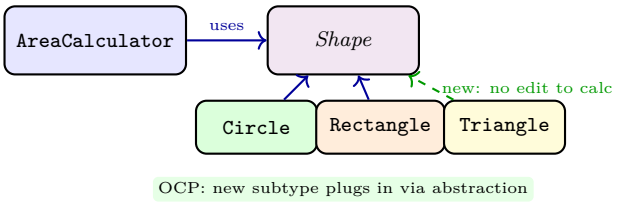


Figure 5.2: OCP in structure: `AreaCalculator` depends on the `Shape` abstraction; adding `Triangle` touches no existing class.

5.2 Liskov, Interface Segregation, Dependency Inversion

LSP—Subtypes must be substitutable. From Ch. 3: overriding must not strengthen preconditions or weaken postconditions. A method expecting `Rectangle` must work with any subtype. Design hierarchies around behaviour, not data shape.

ISP—Many specific interfaces beat one general one. A fat interface forces clients to depend on methods they never use. Split it.

Listing 5.2: ISP: fat interface vs. segregated roles.

```
// Fat: Printer must implement staple() even if it cannot staple
interface Machine { void print(); void scan(); void staple(); }
// Segregated: clients depend only on what they use
interface Printer { void print(); }
```

```
interface Scanner { void scan(); }
interface Stapler { void staple(); }
class SimplePrinter implements Printer { public void print(){ /*...*/ } }
```

DIP—Depend on abstractions, not concretions. High-level policy (e.g., `CheckoutService`) should not depend on low-level detail (`MySQLBookRepository`); both should depend on an abstraction (`BookRepository` interface). This inverts the naive dependency direction.

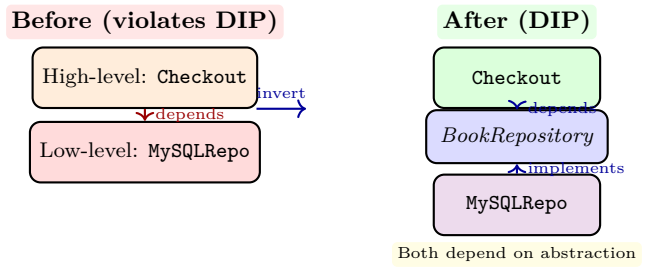


Figure 5.3: DIP: introduce an abstraction so high-level policy does not depend on low-level detail; both depend on the interface.

Remark 5.1 (SOLID together). SRP and ISP split things that change for different reasons; OCP and DIP make extension cheap; LSP keeps hierarchies honest. Apply all five lightly—over-splitting creates indirection without value (YAGNI/KISS). Let the next requirement, not speculation, drive the split.

5.3 How SOLID Feels in Practice: Smells and Heuristics

SOLID is easier to apply when you can name the smell it cures.

Smell	Principle it violates	Fix
God class / long method	SRP	Extract class/method
Switch on type	OCP	Strategy / polymorphism
Subclass throws on super method	LSP	Re-hierarchy or composition
Fat interface, forced empty methods	ISP	Split interfaces
`new` everywhere, hard to test	DIP	Factory + constructor injection

Heuristics that reinforce SOLID. DRY (don't repeat yourself) pushes toward abstraction; YAGNI (you aren't gonna need it) pulls back from speculation—apply SOLID at the second variant, not the first guess. Encapsulation (Ch. 2) and favouring composition over inheritance (Ch. 3) are the mechanisms SOLID builds on.

Listing 5.3: DIP + injection makes testing trivial (preview of Ch. 8).

```
class CheckoutService {
    private final BookRepository repo; // abstraction, not MySQLRepo
    private final PaymentGateway gateway;
    public CheckoutService(BookRepository repo, PaymentGateway gw){
        this.repo = repo; this.gateway = gw; // constructor injection
    }
```

```
    public Receipt checkout(String memberId, String bookId){
        Book b = repo.findById(bookId).orElseThrow();
        // ... policy + payment via gateway
        return new Receipt(b);
    }
}
// Test injects in-memory fakes: new CheckoutService(new InMemoryRepo(), new FakeGateway()
    )
```

When not to SOLID. A one-off script, a prototype, or a module with one known variant does not need five interfaces. Let the pain (second variant, painful test, painful change) justify the abstraction.

5.3.1 From Principles to Code Reviews

A practical review checklist derived from SOLID:

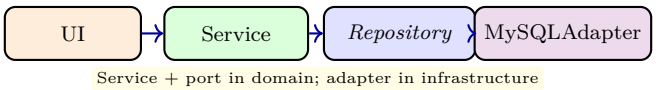
1. Does this class have a one-sentence responsibility? If not, split (SRP).
2. Would a new variant require editing this file? If yes, introduce an abstraction (OCP/DIP).
3. Does any subclass break a supertype test? If yes, fix the hierarchy (LSP).
4. Does any client implement methods it never calls? If yes, split the interface (ISP).
5. Does high-level code mention a low-level concrete name? If yes, invert via interface (DIP).

Apply the checklist on every pull request; the cost is minutes, the saving is weeks of coupled changes. Record the decision (ADR) when you introduce an abstraction so future readers know *why* the seam exists.

Metrics that correlate. Tools like SonarQube flag god classes (SRP), package cycles (DIP), and duplicate switches (OCP). Treat metric warnings as prompts for judgement, not automatic refactor triggers—confirm with the smell table above before changing code.

5.3.2 Dependency Inversion in Layers

In a layered architecture (UI → Service → Repository → DB), naive dependencies point downwards: Service knows MySQL. DIP flips the boundary: Service owns the `Repository` interface in *its* package; the MySQL adapter in the infrastructure package implements it. The dependency arrow now points *inward* toward policy, not outward toward detail—high-level policy is independent of storage choice and can be tested with an in-memory fake.



5.4 Chapter Notes

SOLID was named by Martin (2000) synthesising earlier work (Liskov 1987 for LSP, Martin for DIP/ISP). Effective Java and Clean Architecture (Martin) show pragmatic trade-offs. Coupling/cohesion, DRY, and YAGNI are complementary heuristics from Constantine, Hunt & Thomas, and Beck.

5.5 Exercises

E5.1 **SRP refactor.** `Order` currently validates, calculates total, formats receipt, and saves to DB. Identify each reason to change, split into cohesive classes, and show the new dependencies.

E5.2 **OCP without switch.** A `PaymentProcessor` switches on `type=="CASH"/"CARD"/"PAYNOW"`. Refactor to a `PaymentMethod` hierarchy so adding `CRYPTO` needs no edit to `PaymentProcessor`. Draw the class diagram.

E5.3 **LSP audit.** `ReadOnlyBook extends Book` but throws on `markBorrowed()`. Explain the LSP violation, give a failing client snippet, and propose a hierarchy that honours LSP.

E5.4 **ISP split.** Interface `Worker { work(); eat(); sleep(); }` is implemented by `Robot` (which does not eat/sleep). Apply ISP, show the segregated interfaces, and explain how client coupling drops.

E5.5 **DIP wiring.** `NotificationService` currently constructs `new EmailSender()` internally. Apply DIP with a `Sender` interface, show constructor injection, and explain how testing benefits (mock sender).

Chapter 6

Design Patterns: Reusable Solutions (Strategy, Observer, Factory)

> *"A pattern is a solution to a problem in a context."* — Patterns name recurring design situations and their proven shapes. We study three workhorses—Strategy, Observer, and Factory—deeply enough to apply them, not just recite them.

From zero: built on Ch. 1–5. Each pattern is introduced as a concrete pain (switch, polling, scattered `new`) then solved. Only three patterns, fully worked with UML + Java + sequence traces.

Learning Outcomes

After this chapter you will be able to:

(L1) state the intent, structure, and consequences of Strategy, Observer, and Factory (Method + Simple/Abstract);

(L2) apply Strategy to eliminate switch-on-type and OCP violations;

(L3) apply Observer for event-driven decoupling and explain push vs. pull and leak/lifecycle pitfalls;

(L4) apply Factory to centralise creation, hide concrete types, and honour DIP;

(L5) select among the three patterns (and recognise when *no* pattern is needed).

6.1 Strategy: Encapsulate What Varies

Problem: behaviour varies by case and a `switch` grows with each new case (violates OCP). Example: shipping cost (Standard/Express/Overnight), discount rules, sorting comparators.

Solution: define a *Strategy* interface for the varying behaviour; each variant is a concrete strategy; the *Context* holds a reference to the current strategy and delegates.

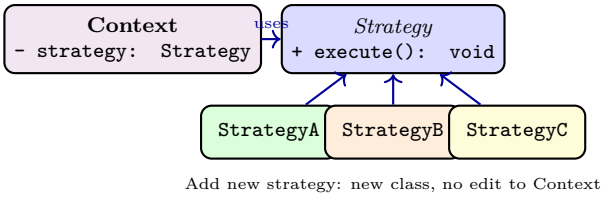


Figure 6.1: Strategy pattern: Context delegates to a Strategy interface; new behaviours are new classes (OCP).

Listing 6.1: Strategy eliminates the switch and honours OCP.

```
interface ShippingStrategy { double cost(double weight); }
class StandardShipping implements ShippingStrategy { public double cost(double w){return
    5+0.5*w;}}
class ExpressShipping  implements ShippingStrategy { public double cost(double w){return
    12+1.2*w;}}
class Checkout {
    private ShippingStrategy shipping;
    public Checkout(ShippingStrategy s){ this.shipping=s; } // inject
    public void setShipping(ShippingStrategy s){ this.shipping=s; }
    public double total(double weight, double items){ return items + shipping.cost(weight)
        ; }
}
// Adding Overnight: new class, no edit to Checkout
class OvernightShipping implements ShippingStrategy { public double cost(double w){return
    25+2.0*w;}}
```

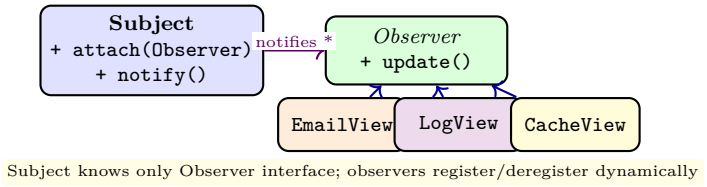


Figure 6.2: Observer: Subject maintains a list of Observers and notifies them on state change. Decouples event source from event consumers.

6.2 Observer: Decouple Event Source from Consumers

Problem: many views depend on one model's state (UI, cache, audit log). Polling is wasteful; direct calls couple model to every view.

Solution: Subject holds a list of `Observers`; on change it calls `notify()` which invokes `update()` on each observer. Observers register/unregister at runtime.

Listing 6.2: Observer with push vs. pull and lifecycle.

```
interface Observer { void update(Book b); } // push: subject pushes data
class BookStore { // Subject
    private List<Observer> observers = new ArrayList<>();
    private List<Book> books = new ArrayList<>();
    public void attach(Observer o){ observers.add(o); }
    public void detach(Observer o){ observers.remove(o); }
    public void addBook(Book b){ books.add(b); notifyObservers(b); }
    private void notifyObservers(Book b){ for(Observer o: observers) o.update(b); }
```

```
}
class AvailabilityView implements Observer {
    public void update(Book b){ System.out.println(b.getTitle()+" now available"); }
}
```

Pitfalls: forgetting to `detach` leaks observers (memory leak, stale updates); notifying while holding a lock can deadlock; ordering of notification may matter. Java provides `PropertyChangeListener` / reactive streams for production use.

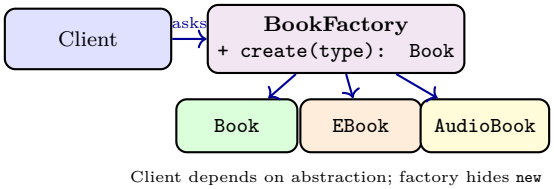


Figure 6.3: Factory: centralise creation so clients depend on abstractions, not concrete constructors. Abstract Factory generalises to families.

6.3 Factory: Centralise Creation

Problem: `new` scattered across the codebase couples clients to concrete classes and duplicates creation logic (validation, caching, subtypes).

Solution: a *Factory* encapsulates creation: `Simple Factory` (one method with a type flag), *Factory Method* (subclasses decide which product to create), *Abstract Factory* (families of related products).

Listing 6.3: Simple Factory and Factory Method.

```
// Simple Factory: one place to change when construction changes
class BookFactory {
    public static Book create(String type, String title){
        return switch(type){
            case "ebook" -> new EBook(title, "https://...");
            case "audio" -> new AudioBook(title, 3600);
            default      -> new Book(title);
        };
    }
}
// Factory Method: subclasses override creation
abstract class LibraryImporter {
    public void importAll(){ Book b = createBook(); /* common logic */ }
    protected abstract Book createBook();
}
class EBookImporter extends LibraryImporter { protected Book createBook(){return new EBook
    ("t","url");}}
```

Factories also enable DIP: high-level code asks the factory for a `Book`, never mentioning `EBook` directly; test code injects a factory that returns mocks.

Remark 6.1 (When not to pattern). If you have one shipping rule and no second one on the horizon, a simple `if` is clearer than Strategy. Patterns pay off at the second or third variant (Rule of Three). Do not apply patterns

speculatively.

6.4 Choosing and Combining Patterns

Decision guide.

- Varying *algorithm/behaviour* chosen at runtime? → **Strategy** (or lambda).
- One-to-many event notification, runtime subscribers? → **Observer**.
- Creation is complex, varies by type/family, or must hide concrete classes? → **Factory**.

Combinations. Patterns compose. A Factory often creates Strategies (`ShippingFactory.create("express")`); an Observer's Subject may be created by a Factory; Strategy objects can be Observers. Recognising composition is more valuable than memorising 23 names.

Listing 6.4: Patterns compose: Factory creates the right Strategy at runtime.

```
class ShippingFactory {
    static ShippingStrategy of(String type){
        return switch(type){
            case "standard" -> new StandardShipping();
            case "express"  -> new ExpressShipping();
            case "overnight"-> new OvernightShipping();
            default -> throw new IllegalArgumentException("unknown: "+type);
        };
    }
}
class Checkout2 {
    double total(String shipType, double w, double items){
        return items + ShippingFactory.of(shipType).cost(w); // OCP + centralised creation
    }
}
```

Modern Java notes. Since Java 8, Strategy is often a `Function` or method reference; Observer is often a `Flow.Publisher` or event bus; Factory benefits from `Supplier<T>` and dependency injection frameworks. The *ideas* outlive the boilerplate.

Remark 6.2 (Anti-patterns). Over-applying patterns is a smell: "FactoryFactory" indirection, Observer chains that become callback hell, and Strategy proliferation where a simple `if` would suffice. Apply the simplest thing that honours the principles (Ch. 5).

6.5 Chapter Notes

Gamma et al. *Design Patterns* (1994) catalogued 23 patterns; we cover three with highest frequency in application code. Head First Design Patterns (Freeman & Robson) has approachable elaborations. Modern Java

often replaces Strategy with lambdas (`Function`) and Observer with `Flow`/reactive streams—same ideas, lighter syntax.

6.6 Exercises

E6.1 **Strategy refactor.** Refactor a `calculateFee(MemberType type)` switch with cases `STUDENT, STAFF, PUBLIC` to Strategy. Show the interface, three strategies, and how `FeeCalculator` is now OCP-compliant.

E6.2 **Observer leak.** An Android Activity registers as observer in `onCreate` but never unregisters. Explain the leak, when it manifests, and fix with `detach` in the lifecycle. How would a weak-reference observer list help?

E6.3 **Push vs. pull.** Compare push (`update(Book b)`) vs. pull (`update()` then `subject.getState()`) for Observer. When does each win? Illustrate with a stock-ticker vs. large document model.

E6.4 **Factory vs. constructor.** A client does `new EBook(...)` in five places. A new requirement adds authentication tokens to every EBook. Show how Simple Factory reduces the change to one place and how it improves testability with a mock factory.

E6.5 **Pattern selection.** For each requirement say which pattern (if any) fits and why: (a) support PayNow, PayLah!, and credit-card payments; (b) notify three UIs when a loan becomes overdue; (c) create families of UI widgets per OS theme (light/dark). Justify with intent matching.

Chapter 7

Exception Handling & Generics: Safety at Compile Time and Runtime

> *"Errors should be impossible by construction; where they remain possible, they should be impossible to ignore."* — Exceptions handle the runtime half; generics handle the compile-time half. Together they make wrong code hard to write and hard to miss.

From zero: built on Ch. 1–5. We define *error*, *exception*, *throw/catch*, *checked vs. unchecked*, and *parametric types* from scratch. No prior generics or exception experience assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish errors, checked/unchecked exceptions, and when to use each;

(L2) write robust try–catch–finally and try-with-resources with correct exception chaining;

(L3) design exception hierarchies and apply fail-fast with meaningful messages;

(L4) declare and use generic classes/methods, bounded wildcards (PECS), and type erasure consequences;

(L5) combine exceptions + generics to build a type-safe, failure-aware API (e.g., `Result<T>` or `Repository<T>`).

7.1 Exceptions: Fail Fast, Fail Clearly

Two bad ways to signal failure: return `null` (caller forgets to check) and return error codes (easy to ignore, no stack trace). Exceptions make failure *unignorable*: if you do not handle it, the program stops at the call site with a trace.

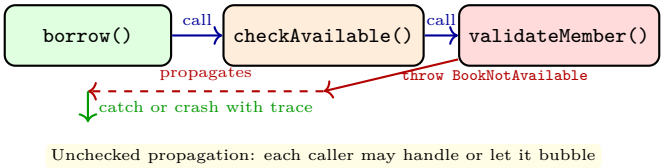
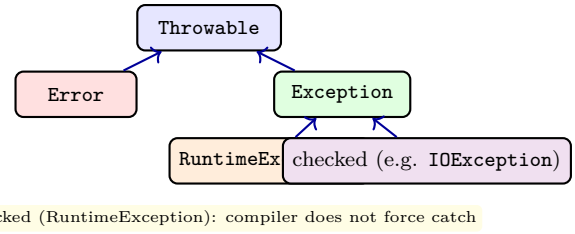


Figure 7.1: Exception propagation: a thrown exception unwinds the call stack until caught; uncaught, it terminates with a stack trace.

Java's hierarchy. `Throwable` → `Error` (JVM, do not catch) and `Exception` → `RuntimeException` (unchecked) vs. other `Exception` (checked). Checked = compiler forces handling; unchecked = programming errors / recoverable runtime failures.



Listing 7.1: Checked vs. unchecked and try-with-resources with chaining.

```
// Custom checked (recoverable) vs unchecked (programming error)
class BookNotAvailableException extends Exception { // checked: caller must handle
    public BookNotAvailableException(String msg){ super(msg); }
}
class InvalidStateException extends RuntimeException { // unchecked
    public InvalidStateException(String msg){ super(msg); }
}
// try-with-resources: AutoCloseable closed even on exception; addSuppressed preserves
    cause
void copy(Path src, Path dst) throws IOException {
    try (InputStream in = Files.newInputStream(src);
         OutputStream out = Files.newOutputStream(dst)) {
        in.transferTo(out);
    } catch (IOException e) {
        throw new IOException("copy failed: " + src + " -> " + dst, e); // chain cause
    }
}
```

Rules: catch the *most specific* type first; never swallow silently (`catch(Exception e){}` hides bugs); prefer try-with-resources over manual `finally` for `Closeables`; chain causes with `new X(msg, cause)` so the root is not lost.

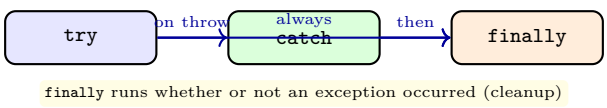


Figure 7.2: Try–catch–finally flow: `try` for guarded code, `catch` per exception type, `finally` for unconditional cleanup.

7.2 Generics: One Definition, Many Types

Without generics, a container holds `Object` and every retrieval needs a cast (unsafe, verbose). Generics let one class/method work for many types with compile-time checking.

Listing 7.2: Generic class and generic method—one definition, type-safe at compile time.

```
// Generic class: Repository works for any T, checked at compile time
class Repository<T> {
    private List<T> items = new ArrayList<>();
    public void add(T t){ items.add(t); }
    public T get(int i){ return items.get(i); } // no cast
}
Repository<Book> books = new Repository<>();
books.add(new Book("Dune")); // ok
// books.add("not a book");  // compile error -- caught early

// Generic method: max works for any Comparable type
<T extends Comparable<T>> T max(T a, T b){ return a.compareTo(b) >= 0 ? a : b; }
```

Invariant generics and wildcards. `List<Book>` is *not* a subtype of `List<LibraryItem>` even if `Book <: LibraryItem` (otherwise you could add an `AudioBook` to a `List<Book>`). Wildcards restore flexibility: `List<? extends LibraryItem>` (producer—read only; PECS: Producer Extends) and `List<? super Book>` (consumer—write).

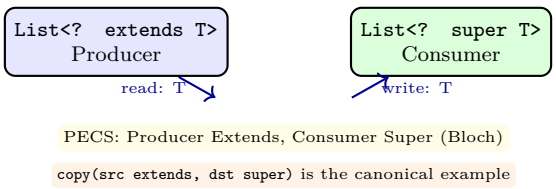


Figure 7.3: PECS for wildcards: producers use `extends` (you read Ts out), consumers use `super` (you write Ts in).

Erasure. Java implements generics by *type erasure*: generic types are checked at compile time then erased to `Object` (or bounds) at runtime. Consequences: cannot `new T()`, cannot `instanceof List<String>`, and primitive types need boxing (`List<Integer>` not `List<int>`). Overloads differing only by generic argument erase to the same signature and are illegal.

7.3 Putting It Together: A Type-Safe, Failure-Aware API

Combine exceptions + generics to design an API that is hard to misuse.

Listing 7.3: Generic Result<T> captures success/failure without null or error codes.

```
sealed interface Result<T> permits Ok, Err {}
record Ok<T>(T value) implements Result<T> {}
record Err<T>(String message, Throwable cause) implements Result<T> {}

```

```
class Library {
    Result<Loan> tryBorrow(String memberId, String bookId){
        try {
            Member m = members.find(memberId);
            Book b = books.find(bookId);
            if (!b.isAvailable()) return new Err<>("book unavailable", null);
            if (!m.canBorrow())   return new Err<>("member limit reached", null);
            Loan ln = Loan.create(m,b);
            return new Ok<>(ln);
        } catch (Exception e){ return new Err<>("borrow failed", e); }
    }
}
```

Why this helps. Callers cannot ignore failure (must handle `Err`), no `null` ambiguity, and the success type is generic (`Result<Loan>` vs. `Result<Book>`) so mixing them is a compile error. For recoverable I/O failures prefer checked exceptions or `Result`; for programming errors prefer unchecked with fail-fast preconditions (`Objects.requireNonNull`, `checkArgument`).

Wildcards in APIs. Use PECS at API boundaries: `void addAll(Collection<? extends T> src)`, `void drainTo(Collection<? super T> dst)`. Inside the implementation, concrete `T` suffices. This maximises caller flexibility without weakening guarantees.

Remark 7.1 (Exceptions vs. Result). Java's idiom is exceptions for exceptional paths and `Optional<T>`/`Result<T>` for expected absence/failure. Pick one style per module and be consistent; mixing both for the same failure confuses callers.

7.3.1 Erasure Pitfalls Worked

Listing 7.4: Erasure pitfalls and workarounds.

```
// Cannot overload on generic type alone -- erases to same signature
// void foo(List<String> a) {} void foo(List<Integer> a) {} // compile error

// instanceof with generics -- test via wildcard + cast with warning suppression
boolean isStringList(Object o){
    if (!(o instanceof List<?> l)) return false;
    return !l.isEmpty() && l.get(0) instanceof String;
}
// Creating arrays of generics -- use collections or Array.newInstance with type token
<T> T[] makeArray(Class<T> tok, int n){
    @SuppressWarnings("unchecked") T[] a =
        (T[])java.lang.reflect.Array.newInstance(tok,n);
    return a;
}
```

Bounded type parameters (`<T extends Comparable<? super T>`) look noisy but enable `max` over any comparable hierarchy. Read them as "T can be compared to itself or a supertype." When the bound is complex, introduce a helper interface to name it.

Nullability and Optional. Generics and `Optional<T>` jointly eliminate many `null` checks: return `Optional<Book>` instead of nullable `Book`, and the compiler forces callers to handle absence. Never use `Optional` as a field type—it is for return values and stream pipelines.

7.3.2 Testing Generics and Exceptions

Test generics with at least two instantiations (`Repository<Book>` and `Repository<Member>`) to catch raw-type leakage. Test exceptions with `assertThrows` for the happy-path failure and with a try-with-resources test that the resource is closed even when an exception is thrown (verify via a spy `Closeable`). For `Result<T>`, test both `Ok` and `Err` branches and that `Err` preserves the cause chain for debugging.

7.4 Chapter Notes

Checked exceptions are a Java distinctive; many languages use only unchecked (Kotlin, C#). Bloch (Effective Java) Items 70–77 cover exception design; Item 26–33 cover generics and PECS. Erasure was a compatibility choice (Java 5); C# reifies generics at runtime.

7.5 Exercises

E7.1 **Checked vs. unchecked.** `BookNotAvailable` on borrow—checked or unchecked? Justify via "can the caller reasonably recover?" Give one example of each in the library domain.

E7.2 **Swallowed exception.** Find the bug: `try{ save(); } catch(Exception e){}`. Show a failure it hides, then fix with specific catch + chaining + logging.

E7.3 **Generics safety.** Without generics, a `List` holding `Book` and `Member` compiles but fails at runtime. Rewrite with `Repository<T>` and show the compile-time error that now prevents the mix-up.

E7.4 **PECS drill.** Write `copy(List<? extends T> src, List<? super T> dst)`. Explain why `src` is `extends` and `dst` is `super` and what goes wrong if you swap them.

E7.5 **Erasure consequence.** Explain why `new T()` and `instanceof List<String>` are illegal after erasure. Propose a workaround for each (factory/type token).

Chapter 8

Testing & Refactoring: Keeping Code Correct and Clean

> *"Refactoring without tests is not refactoring."* — Tests are the safety net that makes change cheap. This chapter teaches you to test behaviour (not implementation), to refactor structure while preserving behaviour, and to know when to do each.

From zero: built on Ch. 1–7. We define *test*, *oracle*, *coverage*, *refactor*, and *code smell* from scratch. Tool: JUnit 5 + Mockito (concepts port to any framework). No prior testing assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) write JUnit 5 tests (arrange–act–assert), use fixtures, and distinguish unit vs. integration tests;

(L2) apply equivalence partitioning and boundary-value analysis to choose minimal, high-yield test cases;

(L3) use test doubles (stub, mock, spy) and explain when to mock and when not to;

(L4) refactor safely via small, behaviour-preserving steps (extract method/class, replace conditional with polymorphism, etc.);

(L5) measure and interpret coverage without gaming it, and run a TDD red–green–refactor cycle.

8.1 Testing: The Safety Net

Tests *specify* expected behaviour and *detect* regressions. A good test is fast, isolated (no shared mutable state), and asserts on observable behaviour, not internal wiring.

Listing 8.1: JUnit 5 basics—arrange, act, assert; fixture with BeforeEach.

```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;

class BookTest {
    private Book book;
    @BeforeEach void setUp(){ book = new Book("Dune"); } // fixture
    @Test void newBookIsAvailable(){ assertTrue(book.isAvailable()); }
    @Test void borrowMakesUnavailable(){
        book.markBorrowed();
        assertFalse(book.isAvailable());
    }
    @Test void doubleBorrowThrows(){
        book.markBorrowed();
        assertThrows(IllegalStateException.class, () -> book.markBorrowed());
    }
    @Test @DisplayName("toString contains title") void toStringHasTitle(){
        assertTrue(book.toString().contains("Dune"));
    }
}
```

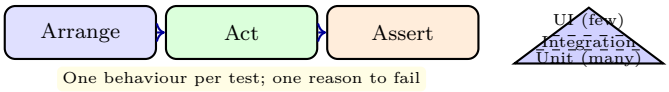


Figure 8.1: Left: Arrange–Act–Assert (one behaviour per test). Right: test pyramid—many fast unit tests, fewer integration/UI tests.

Choosing test cases. No exhaustive testing. Use *equivalence partitions* (inputs that should behave the same) and *boundary values* (edges: 0, 1, MAX, null, empty). For `canBorrow` with limit 5: test 0, 4, 5, 6 loans; for `dueDate`: on-time, one day late, far overdue.

Test doubles. When a unit depends on slow/fragile collaborators (DB, network), replace them: *stub* (returns canned data), *mock* (verifies interactions), *spy* (wraps real object), *fake* (lightweight working implementation). Mock only across boundaries you own—over-mocking couples tests to implementation.

Listing 8.2: Mocking a collaborator—verify interaction, not just state.

```java
import static org.mockito.Mockito.*;

class CheckoutTest {
    @Test void borrowDelegatesToBook(){
        Book mockBook = mock(Book.class);
        when(mockBook.isAvailable()).thenReturn(true);
        Member m = new Member("u1", 5);
        Loan loan = Loan.create(m, mockBook);
        verify(mockBook).markBorrowed(); // interaction asserted
        assertNotNull(loan);
    }
}
```

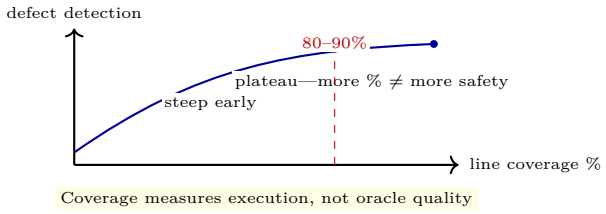


Figure 8.2: Coverage vs. confidence: early coverage finds real gaps; chasing 100% often tests trivial code while missing semantics.

8.2 Refactoring: Change Structure, Preserve Behaviour

A *refactor* is a behaviour-preserving transformation that improves structure (readability, cohesion, coupling). Refactors are small and sequenced; tests stay green after each step. Big-bang rewrites are not refactors.

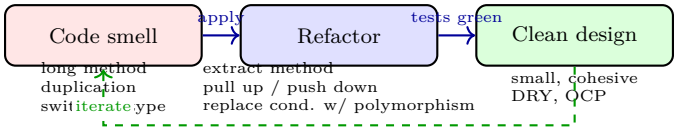


Figure 8.3: Refactoring loop: detect a smell, apply a small behaviour-preserving refactor, verify with tests, repeat.

Common refactors and the smells they cure:

- *Extract Method/Class* ← long method / god class (SRP).
- *Replace Conditional with Polymorphism* ← switch-on-type (OCP)—introduce Strategy/hierarchy from Ch. 6.
- *Move Method* ← feature envy (method uses another class's data more than its own).
- *Introduce Parameter Object / Builder* ← long parameter list.

Listing 8.3: Refactor: replace switch with polymorphism (Ch. 5–6 applied).

```
// Before: violates OCP, hard to test
double shippingCost(String type, double w){
    switch(type){ case "standard": return 5+0.5*w; case "express": return 12+1.2*w;
        default: throw new IllegalArgumentException(); }
}
// After: each variant is a class; adding a variant touches no existing class
interface ShippingStrategy{ double cost(double w); }
class StandardShipping implements ShippingStrategy{ public double cost(double w){return
    5+0.5*w;}}
class ExpressShipping  implements ShippingStrategy{ public double cost(double w){return
    12+1.2*w;}}
// Client now depends on abstraction (DIP), tests inject a stub strategy
```

8.2.1 TDD: Red–Green–Refactor

Test-Driven Development inverts the order: write a failing test (red), write minimal code to pass (green), refactor with safety. Benefits: tests actually get written, design stays testable, scope stays minimal. Trade-off:

not suited for exploratory spikes where the interface is unknown.

Remark 8.1 (Coverage guidance). Aim for high coverage on domain logic; do not chase 100% by testing getters/setters. Mutation testing (PIT) is a stronger signal: it checks whether tests *detect* injected faults, not just execute lines.

8.3 Refactoring Catalogue in Detail

Each refactor below is a small, named, behaviour-preserving step. Apply one at a time and run tests after each.

Extract Method. Long method with a coherent chunk → new private method with a descriptive name. Improves readability and enables reuse. Precondition: extracted code has no hidden side-effects on local control flow.

Extract Class / Move Method. Method on `A` that uses `B`'s data more than `A`'s → move it to `B` (feature envy). Class with two axes of change → split into two classes (SRP).

Replace Conditional with Polymorphism. Switch/if-chain on type tag → introduce a Strategy/hierarchy (Ch. 6). OCP payoff: new variants add classes, not edits.

Introduce Parameter Object / Builder. Method with 4+ parameters or telescoping constructors → bundle into a parameter object or Builder. Reduces call-site errors and documents the parameter set.

Listing 8.4: Introduce Parameter Object + Builder for a telescoping constructor.

```
// Before: confusing, error-prone call
Loan l = new Loan(member, book, dueDate, true, false, null);
// After: self-documenting, validated at build time
Loan l2 = Loan.builder().borrower(member).book(book).dueDate(dueDate).build();
```

8.3.1 TDD Traces and When Not to Mock

TDD trace for `Money`: (1) red: `Money.of(10,"SGD").plus(Money.of(5,"SGD")) == Money.of(15,"SGD");` (2) green: naive add; (3) refactor: handle currency mismatch → throw. Each cycle is minutes, not hours.

Mock sparingly: mock I/O boundaries (repository, gateway), not value objects (`Money`, `Isbn`) or stable domain entities. Over-mocking makes tests brittle to internal renames.

Remark 8.2 (Safe refactoring checklist). Before refactoring: green tests, small commit. During: one refactor per commit, no behaviour change. After: tests still green, review for accidental semantics change (especially around equals/hashCode and exception paths).

8.4 Chapter Notes

Refactoring catalogue is Fowler (1999/2018); TDD is Beck (2002). JUnit 5 User Guide and Mockito docs are the practical references. Coverage vs. mutation is discussed in Effective Software Testing (Maurer & Scheffczyk). Clean Code (Martin) covers smell naming.

8.5 Exercises

E8.1 **AAA audit.** A test has 30 lines, 4 assertions, and touches DB + filesystem. Diagnose using AAA, isolation, and pyramid guidance, then split it into focused unit tests with fakes/mocks.

E8.2 **Partitions & boundaries.** For "loan period 14 days, fine $1/day after 7-day grace, max fine $30," list equivalence partitions and boundary values. Write the minimal JUnit cases that cover them.

E8.3 **Mock boundary.** `Checkout.borrow` calls `Book.isAvailable()` and `Member.canBorrow()`. Which collaborators should be mocked in a unit test of `Checkout` and which should be real? Justify with coupling and oracle strength.

E8.4 **Refactor sequence.** Apply "Replace Conditional with Polymorphism" to the shipping-cost switch in the listing. List the steps in order (create interface, extract strategies, update client, run tests) and state what could go wrong if done as one big edit.

E8.5 **TDD cycle.** Using TDD, implement `Money` (immutable, currency-aware, add/subtract with same currency, no floating point). Show the red–green–refactor sequence for the first three tests and the design decisions each forced.